

Software Design Document

Owl Eye Monitoring System

Prepared By:

Ian Riggins, Isa Siddique, Evan Valencia, William Walker, Huy
Nguyen

TEAM 25 - 02 | Kennesaw State University

TABLE OF CONTENTS

Table of Contents.....	ii
1) Introduction.....	1
1.1) Purpose.....	1
1.2) Scope.....	1
1.3) Overview.....	1
1.4) Reference Material.....	1
1.5) Definitions and Acronyms.....	1
2) System Overview.....	3
3) System Architecture.....	4
3.1) Architectural Design.....	4
3.2) Technical Design.....	4
3.3) Design Rationale.....	4
4) Detailed Design.....	6
5) Data (Database) Design.....	7
6) Human Interface Design.....	8
6.1) UI Design.....	8
6.2) UX Design.....	10
7) Requirements Matrix.....	13
8) Appendices.....	15

1) INTRODUCTION

1.1)PURPOSE

This Software Design Document describes the architecture and system design of the Owl Eye Monitoring System and the supplementary Troublemaker that will be used to test the system via stress testing.

1.2)SCOPE

The OwlEye Monitor system is designed to ensure the continuous health of applications and infrastructure by leveraging modern DevOps practices. Its primary purpose is to simulate real-world server anomalies through the Troublemaker application, which generates events such as CPU and memory load spikes, directory overflows, log errors, and simulated MySQL database error transactions. The system is operated on a dedicated Linux server and is integrated with a Grafana-based monitoring GUI to showcase alerts, send notifications, and attempt auto-resolution of issues. Sponsored by KSU and developed as part of a practical learning initiative, the project brings together system administrators, developers, and students to implement and maintain robust monitoring practices.

1.3)OVERVIEW

This document provides a detailed layout of the OwlEye Monitoring System project's design and architecture. This document will include details about the Monitoring/Alerting System as well as the supplementary Troublemaker Application that will be used for testing.

1.4)REFERENCE MATERIAL

This document does not currently reference any currently existing material.

1.5)DEFINITIONS AND ACRONYMS

Term:	Definition:
FDD	FDD stands for Feature-driven development. It is an Agile model of software development that utilizes a series of techniques in order to meet its development goals. Techniques include creating feature lists, constructing an entire model, designing, developing, and building through features.
KSU	Kennesaw State University

SQL	SQL stands for Structured Query Language. It is a programming language that is used to manage and manipulate relational databases.
-----	--

2) SYSTEM OVERVIEW

2.1) FUNCTIONALITY

The OwlEye Monitor project is a DevOps monitoring and alerting system that detects and responds to system anomalies. The system will consist of a troublemaker application that will generate artificial system stress events along with a monitoring and alerting system that will trigger an alert after observing these anomalies.

2.2) CONTEXT

This project aims to simulate real-world scenarios by introducing controlled stress events and providing students with hands-on experience in setting up a monitoring infrastructure. By working on this project, students will gain exposure to key DevOps tools like Prometheus, Grafana, and AlertManager, while also learning about automated remediation strategies.

2.3) DESIGN

The OwlEye Monitor system consists of the following components:

- **Troublemaker Application:** Generates CPU and memory stress, directory overflows, log errors, and MySQL transaction failures.
- **Monitoring System:** Uses **Prometheus** to collect system metrics and **Grafana** for real-time visualization.
- **Alerting Mechanism:** Implements Prometheus AlertManager to detect abnormal conditions and send notifications via email or messaging services.

3) SYSTEM ARCHITECTURE

3.1) ARCHITECTURAL DESIGN

This system will be divided into 2 modules.

3.1.1) Troublemaker Application

The troublemaker module is responsible for generating, scheduling, and simulating a multitude of various stress events on the server.

3.1.2) OwlEye Monitoring and Alerting System

The monitoring and alerting module is responsible for monitoring the server in real-time and detecting stress events generated by the troublemaker application. Additionally, when high stress events are detected, the system should send alerts via email notifying recipients of the occurrence.

3.2) TECHNICAL DESIGN

Technology	Purpose
Prometheus	Collects and stored system metrics such as CPU, memory, and network usage.
Grafana	Provides real-time visualization of collected metrics.
AlertManager	Handles alerts and notifications.
Troublemaker Application	Simulates system stress (CPU spikes, memory leaks, etc.)

Troublemaker Application:

- Generates artificial stress conditions
- Interacts with database systems to simulate transaction failures
- Logs failures and sends them to Prometheus for monitoring

Monitoring System:

- Prometheus collects metrics from system components
- Grafana provides real-time visualization of system health

Alerting Mechanism:

- Process alerts generated by Prometheus
- Send notifications via email
- Supports alert deduplication, routing, and escalation.

3.3) DESIGN RATIONALE

Scalability & Performance -

- Prometheus is optimized for high-performance time-series data collection, making it ideal for large-scale monitoring.
- Grafana provides an efficient visualization layer without introducing unnecessary processing overhead.

Fault Tolerance & Resilience -

- The system is designed to handle failures gracefully by using AlertManager to route alerts and reduce noise.
- The Troublemaker Application ensures that failures are tested proactively.

Flexibility & Extensibility -

- The architecture allows easy integration with cloud-based deployments.
- Custom alerts can be configured dynamically based on real-time conditions.

Automation & DevOps Alignment -

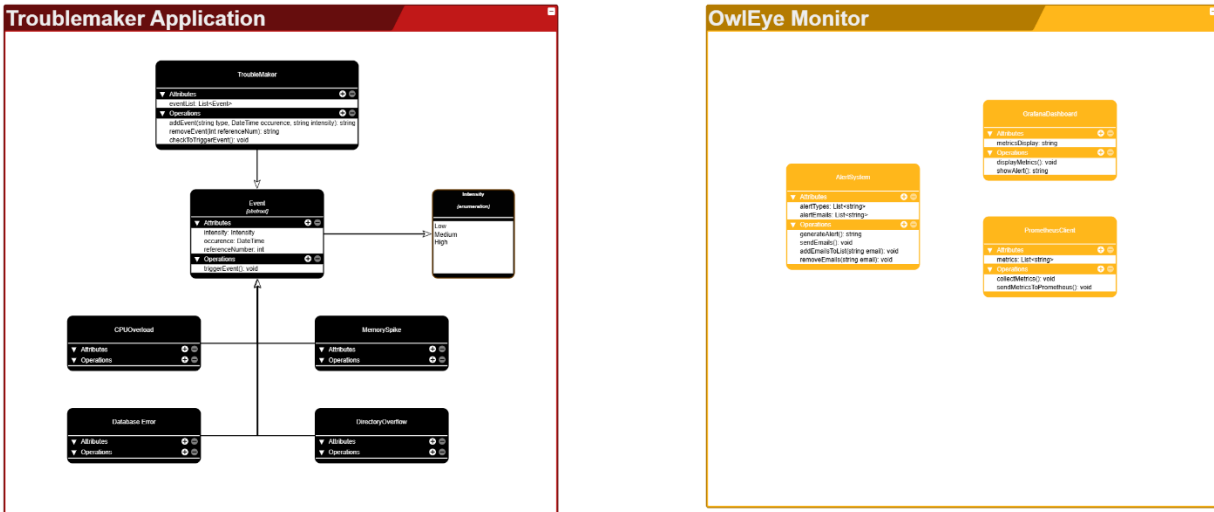
- The selected stack aligns with modern DevOps best practices, where observability, automation, and proactive failure detection are essential.

Critical Issues Considered:

- Avoiding Alert Fatigue
 - Challenge: Too many alerts can overwhelm teams, reducing the effectiveness of monitoring.
 - Solution: AlertManager handles alert deduplication and routing, ensuring only relevant alerts are sent.
- Real-Time vs. Batch Processing
 - Trade-off: A real-time system provides faster insights but can introduce more computational overhead.
 - Decision: Chose real-time monitoring with Prometheus instead of batch processing to ensure immediate alerting.

4) DETAILED DESIGN

As previously mentioned, this system will be separated into the OwlEye (Monitoring and Alerting) System and the supplementary Troublemaker application. Below is a preliminary class diagram detailing the system and its architecture.



The Troublemaker Application will contain a List/ArrayList of various events which represent attacks on the system. Events can vary in intensity, occurrences, and type of attacks but all events and methods will be overridden from the base abstract class.

The OwlEye System will utilize Prometheus Cloud to monitor the system and Grafana to generate and display visualization of the metrics. An alerting system will also be developed which when prompted, will send out an alert email to a configurable List/ArrayList of emails detailing the type of attack and its severity. The prompt will be activated after Prometheus detects that a certain threshold has been exceeded.

5) DATA (DATABASE) DESIGN

Data Transformation and Storage:

Troublemaker Application

- Purpose: Simulates CPU, memory, and MySQL failures.
- Transformation: Generates stress events that are converted into Prometheus metrics.
- Data Storage:
 - Temporary logs (stored locally before being processed).
 - Metrics exported to Prometheus in time-series format.
 - Errors logged in a MySQL database for failure analysis.

Monitoring System

- Purpose: Stores and processes real-time performance metrics.
- Transformation:
 - System performance data (CPU, Memory, Disk) is stored as time-series data points.
 - Grafana queries Prometheus for visualization.
- Data Storage:
 - Prometheus Time-Series Database (stores historical system metrics).
 - Grafana Dashboard Configurations (JSON-based storage for UI visualization).

Alerting System

- Purpose: Detects anomalies and sends notifications.
- Transformation:
 - When a metric crosses a threshold, an alert is triggered.
 - The alert is processed, deduplicated, and routed to the right recipient.
- Data Storage:
 - AlertManager Configuration Files (YAML-based storage defining alerting rules).
 - Notification Logs (stored in a local DB for auditing).

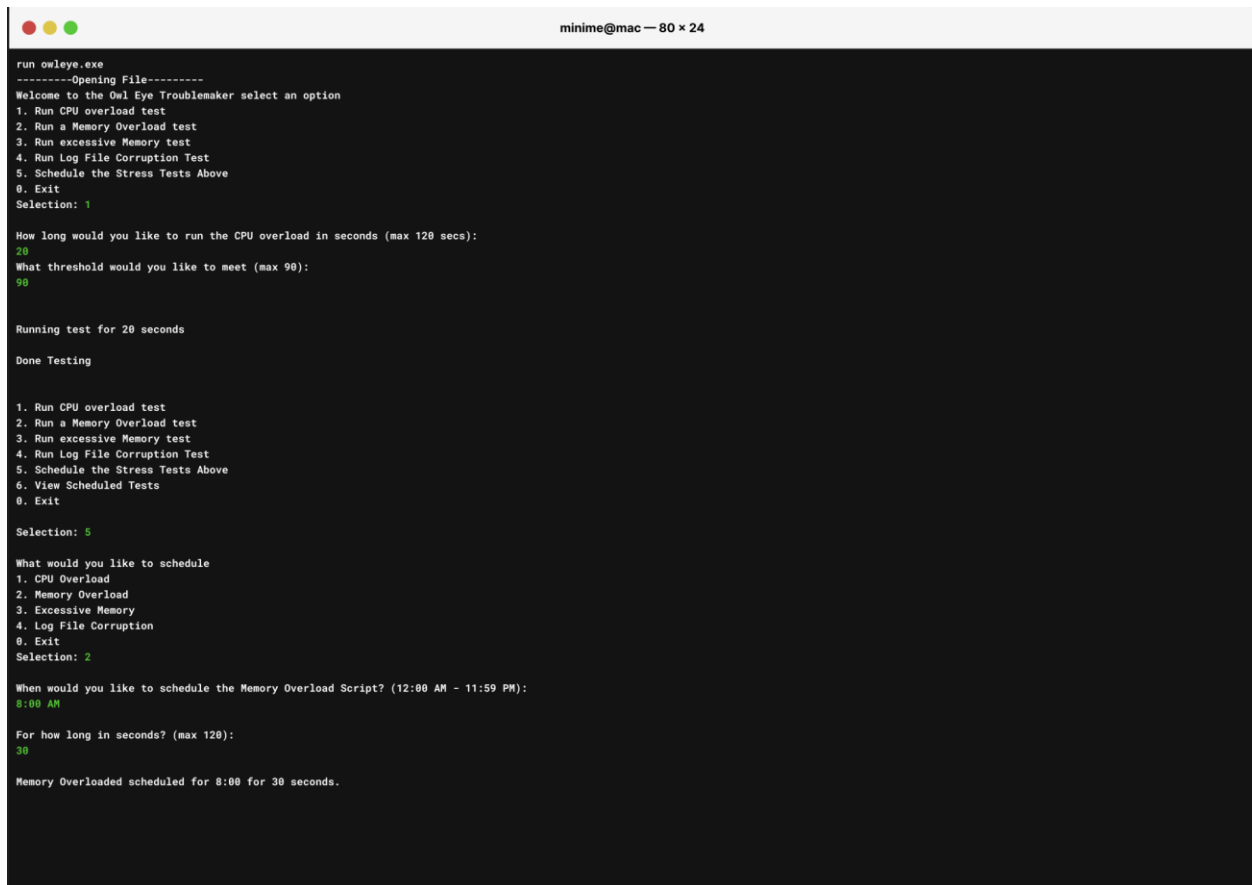
Data Storage Components:

Storage Component	Purpose
Prometheus DB	Stores metrics (CPE, memory, network usage)
Grafana Configurations	Stores dashboard settings and visualization queries
SQL Database	Stores historical error logs and system failures
AlertManager Logs	Stores alert history and notifications for auditing

6) HUMAN INTERFACE DESIGN

6.1) UI DESIGN

Troublemaker Application with Terminal Design. The terminal design was chosen to accomidate the lack of technical skill in Linux application design.

A screenshot of a terminal window titled "minime@mac — 80 x 24". The terminal displays the output of running "owleye.exe". The application starts with a welcome message and a list of options: 1. Run CPU overload test, 2. Run a Memory Overload test, 3. Run excessive Memory test, 4. Run Log File Corruption Test, 5. Schedule the Stress Tests Above, 0. Exit. The user selects option 1. The application then asks for the duration of the CPU overload test (max 120 seconds). The user enters 20. Next, it asks for a threshold (max 90). The user enters 90. The application then runs the test for 20 seconds and displays "Done Testing". It then shows the same menu of options. The user selects option 5. The application asks what to schedule: 1. CPU Overload, 2. Memory Overload, 3. Excessive Memory, 4. Log File Corruption, 0. Exit. The user selects option 2. It then asks when to schedule the Memory Overload Script (12:00 AM - 11:59 PM). The user enters 8:00 AM. Finally, it asks for how long in seconds (max 120). The user enters 30. The application concludes by displaying "Memory Overloaded scheduled for 8:00 for 30 seconds."

```
run owleye.exe
-----Opening File-----
Welcome to the Owl Eye Troublemaker select an option
1. Run CPU overload test
2. Run a Memory Overload test
3. Run excessive Memory test
4. Run Log File Corruption Test
5. Schedule the Stress Tests Above
0. Exit
Selection: 1

How long would you like to run the CPU overload in seconds (max 120 secs):
20
What threshold would you like to meet (max 90):
90

Running test for 20 seconds

Done Testing

1. Run CPU overload test
2. Run a Memory Overload test
3. Run excessive Memory test
4. Run Log File Corruption Test
5. Schedule the Stress Tests Above
6. View Scheduled Tests
0. Exit
Selection: 5

What would you like to schedule
1. CPU Overload
2. Memory Overload
3. Excessive Memory
4. Log File Corruption
0. Exit
Selection: 2

When would you like to schedule the Memory Overload Script? (12:00 AM - 11:59 PM):
8:00 AM

For how long in seconds? (max 120):
30

Memory Overloaded scheduled for 8:00 for 30 seconds.
```

```
minime@mac — 80 x 24

What would you like to schedule
1. CPU Overload
2. Memory Overload
3. Excessive Memory
4. Log File Corruption
0. Exit
Selection: 0

1. Run CPU overload test
2. Run a Memory Overload test
3. Run excessive Memory test
4. Run Log File Corruption Test
5. Schedule the Stress Tests Above
0. Exit
Selection: 5

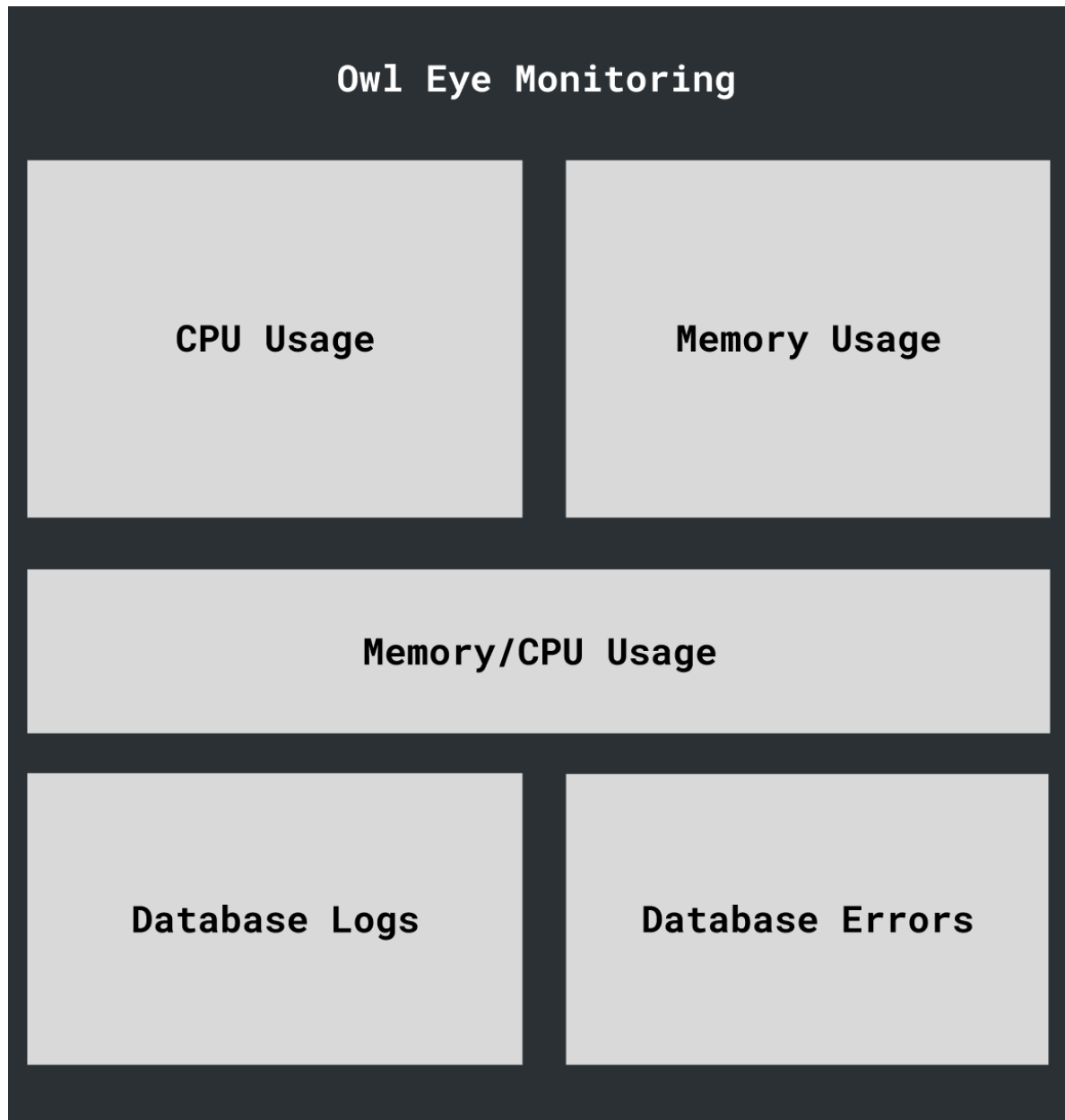
Showing Scheduled Stress tests if you want to remove any type the number next to it:
1. Memory Test at 8:00 AM for 30 seconds
2. CPU Test at 7:45 AM for 50 seconds
3. Excessive Memory Test at 3:00 pm for 120 seconds
4. File Corruption Test at 8:00 pm for 20 seconds
0. Exit
Remove : 2

1. Memory Test at 8:00 AM for 30 seconds
2. CPU Test at 7:45 AM for 50 seconds
3. Excessive Memory Test at 3:00 pm for 120 seconds
4. File Corruption Test at 8:00 pm for 20 seconds
0. Exit
Remove : 0

1. Run CPU overload test
2. Run a Memory Overload test
3. Run excessive Memory test
4. Run Log File Corruption Test
5. Schedule the Stress Tests Above
0. Exit
Selection: 0

Bye Bye
Owl Eye Shut Down
```

Grafana Design:



6.2)UX DESIGN

Troublemaker Application using UX principles, where there is no worry about team technical skills in Linux:

OwlEye Troublemaker

OwlEye Troublemaker Logo

Test Type

Duration

20

+

-

seconds

Threshold

20

+

-

%

At What Time

09:09

AM

PM

Test

Schedule

OwlEye Troublemaker

OwlEye Troublemaker Logo

Test Type

Duration

20

+

-

seconds

Threshold

20

+

-

%

Schedule

Test

7) REQUIREMENTS MATRIX

Requirement ID	Functional Requirement	System Component	Relevant Data Structure
FR-1	Detect and monitor system anomalies	Monitoring System (Prometheus, Grafana)	Metrics_collector
FR-2	Generate artificial system stress events	Troublemaker Application	Troublemaker, CPUStress, MemoryLeak, DatabaseError, LogOverflow
FR-3	Trigger alerts upon detecting anomalies	Alerting Mechanism (AlertManager)	AlertManager, Notification
FR-4	Provide real-time visualization of system	Monitoring System (Grafana)	metrics_collector

	metrics		
FR-5	Log detected anomalies for further analysis	Monitoring System (Prometheus)	Metrics_log
FR-6	Send notifications upon critical alerts	Alerting Mechanism (AlertManager)	AlertManager, Notification
FR-7	Support integration with MySQL for tracking failures	Troublemaker Application	DatabaseError
FR-8	Handle various types of system stress conditions	Troublemaker Application	CPUStress, MemoryLeak, LogOverflow, DatabaseError

8) APPENDICES

This section is optional.

Appendices may be included, either directly or by reference, to provide supporting details that could aid in the understanding of the Software Design Document.